FailMem Supplementary Materials

FailMem: Supplementary Materials & Verification Audit Details

Title: Auditable Failure Memory for ROS 2 Navigation: Supplementary Materials
Document Status: Accompanying Material for Review & Reproduction
Associated Commit: `e3cce9d`

S1. Precondition Integrity & Doorway Clearance Details

The passage state evaluation in `src/doorway_evaluator.py` enforces strict sensor-TF alignment rules prior to emitting a clearance classification:

S1.1 Precondition Rules

1. **Finite Transforms**: TF translation vector $\mathbf{t} = [x, y, z]^T$ and yaw angle ψ must be finite real numbers ($-\infty < x, y, z, \psi < \infty$).
2. **Temporal Alignment**: Scan and TF timestamps must satisfy $|t_{\text{tf}} - t_{\text{scan}}| \leq 0.80\,\text{s}$.
3. **Data Freshness**: Current simulation clock t_{sim} must satisfy $t_{\text{sim}} - t_{\text{tf}} \leq 1.0\,\text{s}$ and $t_{\text{sim}} - t_{\text{scan}} \leq 1.5\,\text{s}$.
4. **Ray Projections**:
   - Number of finite laser scan ranges $N_{\text{valid}} > 0$.
   - Number of projected rays intersecting the doorway bounding volume $N_{\text{intersect}} > 0$.
5. **Costmap Availability**: When costmap subgrids are provided, total cells in the Region of Interest (ROI) $N_{\text{total_cells}} > 0$.

If any precondition fails, the evaluator emits $\text{State}(B) = \text{UNKNOWN}$ and does not alter memory state.

S1.2 Bounding Volume & Threshold Parameters

- Doorway center: $x = 0.0\,\text{m}, y = 1.20\,\text{m}$.
- ROI bounding box: $x \in [-0.15, 0.15]\,\text{m}, y \in [0.95, 1.45]\,\text{m}$.
- Minimum pass-through rays for `FREE`: $N_{\text{pass_through}} \geq 8$.
- Maximum hits inside for `FREE`: $N_{\text{hits_inside}} = 0$.
- Hits threshold for `OCCUPIED`: $N_{\text{hits_inside}} \geq 5$ or $N_{\text{costmap_occupied_cells}} > 0$.

S2. Cryptographic Replay Audit Engine

To avoid reliance on self-reported runtime metrics, the independent replay auditor (`scripts/replay_and_score_p2c.py`) re-scores every physical simulation run offline:

S2.1 Replay Integrity Verification Steps

1. **Manifest Audit**: Verifies every serialized file against `checksums.sha256`.
2. **Trajectory Distance**: Computes traversed distance by summing Euclidean distances between consecutive odometry sample positions $\langle x_k, y_k \rangle_{k=1}^{K}$:

$$d = \sum_{k=2}^{K} \sqrt{(x_k - x_{k-1})^2 + (y_k - y_{k-1})^2}$$

3. **Halt Stability Verification**: Verifies that upon goal completion, the robot remains stationary over a minimum nominal window of $2.0\,\mathrm{s}$:

$$\max_{t\in[t_{\mathrm{goal}},t_{\mathrm{goal}}+\Delta t_{\mathrm{stab}}]}|v_{\mathrm{lin}}(t)|\le 0.05\,\mathrm{m/s},\qquad \max_{t\in[t_{\mathrm{goal}},t_{\mathrm{goal}}+\Delta t_{\mathrm{stab}}]}|v_{\mathrm{ang}}(t)|\le 0.08\,\mathrm{rad/s}$$

4. **Chronological Causality**:

$$t_{\mathrm{rec}}\ge\max(t_{\mathrm{action_end}},t_{\mathrm{obs_eval}})-0.05\,\mathrm{s}$$

$$t_{\mathrm{inv}}\ge\max(t_{\mathrm{failure}},t_{\mathrm{free_eval}})-0.05\,\mathrm{s}$$

S3. Machine-Independent PDF Build Pipeline

The manuscript build pipeline (`paper/scripts/build_paper_pdf.py`): 1. **Dynamic Table Injection**: Substitutes `<!-- TABLE:... -->` markers with freshly generated LaTeX and Markdown tables. 2. **MathJax Vector SVG Pre-Rendering**: Converts LaTeX formulas into inline SVG vector paths via Node.js `mathjax-full@3.2.2` resolved from project root `package.json`. 3. **Table Row Validation**: Verifies generated HTML table structure prior to PDF compilation: - Table 1 (Condition Summary): Exactly 10 data rows. - Table 2 (Pairwise Contrasts): Exactly 10 data rows. - Table 3 (H_1 Feasibility): Exactly 4 data rows. 4. **Zero Raw TeX Leak Assertion**: Renders compiled PDF pages to PNG at $150\,\mathrm{dpi}$ (`pdftoppm`) and extracts text (`pdftotext`); raises `RuntimeError` on unrendered TeX control sequences. 5. **Structured Build Record**: Emits `paper/build_report.json` documenting tool versions, pre-build git status, post-build workspace changes, and cryptographic input/output digests.

S4. Detailed Parameter & Source Code Traceability

For exact code symbol and line-number mappings for all mathematical formulations and protocol parameters, refer to `paper/method-code-map.csv` (33 parameter mappings). For the complete literature primary verification ledger, refer to `paper/reference-verification.csv` (15 primary-source verified references).